1. Базовые типы данных и операции над ними

1. Назовите изменяемые и неизменяемые типы данных в Python.
2. Чем отличаются списки (`list`) от кортежей (`tuple`)?
3. Какая операция в Python возвращает остаток от деления? Приведите пример.
4. Как явно преобразовать `float` в `int`? Что при этом происходит с дробной частью?
5. Какие основные методы работы со списками вы знаете? (например, `.append()`, `.extend()`, `.insert()`, `.pop()`)
6. Как объединить два списка? Продемонстрируйте оба способа: через оператор `+` и через метод.

2. Словари (`dict`) и множества (`set`)

1. Как получить значение по ключу с помощью метода `.get()`? Что возвращается, если ключ отсутствует?
2. Как получить список всех ключей/значений словаря?
3. Как объединить два словаря в новый, при этом перезаписав общие ключи?
4. Как из двух списков получить словарь, где в качестве ключей идут элементы первого списка, а в качестве значений — элементы второго?
5. Как из списка удалить дубликаты, используя `set`?
6. В чем разница между агрегацией и композицией в терминах ООП? (набросайте аналогию со словарями/списками)

3. Условные конструкции (`if` / `elif` / `else`)

1. Как проверить, что число `n` положительное и кратно 3?
2. Продемонстрируйте вложенный `if` — приведите пример.
3. Как проверить, находится ли элемент `x` в списке `lst`?
4. Как в одном выражении проверить и тип (например, `int`), и значение переменной?
5. Чем отличается `if … except` –проверка от `try…except`? В каких случаях что применять?

4. Циклы (`for`, `while`) и управление ими

1. Как перебирать ключи и значения словаря в цикле `for`?
2. Напишите цикл, который выводит только элементы строкового списка длины > 5.
3. Как досрочно прервать `while`-цикл? В чем разница между `break` и `continue`?
4. Демонстрируйте вложенный цикл по списку списков.
5. Как с помощью цикла `while` вывести числа от 1 до 10000? Почему для таких задач иногда используют `for` быстрее?

5. Функции

1. Чем отличается функция без параметров от функции с параметром по умолчанию?
2. Как в теле функции вернуть значение? Как обработать ситуацию, если ветвей `return` несколько?
3. Как вызвать одну функцию внутри другой? Приведите пример.
4. Напишите функцию, принимающую список чисел, и возвращающую сумму элементов.
5. Объясните алгоритм пузырьковой сортировки и оцените его временную сложность.
6. Напишите (или опишите) функцию `bubble_sort(lst)` на словах: как устроены внешние и внутренние циклы, когда можно досрочно завершить сортировку.

6. Объектно-ориентированное программирование

1. Перечислите базовые принципы ООП (абстракция, инкапсуляция, наследование, полиморфизм). Приведите краткую характеристику.
2. Зачем нужны приватные поля (`__attr`), геттеры и сеттеры? Как их реализовать в Python?
3. Опишите, как вы реализовали класс `Bus`, содержащий список объектов `Child`, и методы добавления/удаления ребёнка.
4. Как в методе автобуса изменить местоположение сразу у всех детей?
5. Объясните разницу между наследованием и композицией на примере вашей домашки: класс «Человек» → класс «Ребёнок» → класс «Автобус».

7. Обработка исключений

1. Как устроена конструкция `try…except…else…finally`? В каком порядке выполняются блоки?
2. Чем отличается `except Exception:` от `except:` без аргументов? Почему «голый» `except` лучше не использовать?
3. Как определить своё исключение? Напишите пример класса исключения, унаследованного от `ValueError`.
4. Как в вашем проекте по «роте» и «полку» вы отлавливали ошибки при добавлении/удалении?
5. Когда имеет смысл делать досрочный выход из `try`-блока через флаг (как в пузырьковой сортировке)?